

WaferLLM：晶圆尺度的大语言模型推理

Congjie He¹ Yeqi Huang¹ Pei Mu¹ Ziming Miao²

Jilong Xue² Lingxiao Ma² Fan Yang² Luo Mai¹

¹ 爱丁堡大学 ² 微软研究院

arXiv:2502.04563 (中文翻译版)

摘要

新兴 AI 加速器日益采用晶圆级制造技术：它们以网格架构集成数十万个 AI 核心，同时提供大容量的分布式片上内存（总计数十 GB）和极高的片上内存带宽（数十 PB/s）。然而，现有 LLM 推理系统针对 GPU 等共享内存架构进行了优化，无法充分发挥这些加速器的能力。

本文提出首个晶圆级 LLM 推理系统 WaferLLM。WaferLLM 由一种新的 PLMR 模型（读作“Plummer”）指导，该模型刻画了晶圆级架构独特的硬件特性。在这一模型的指导下，WaferLLM 首创晶圆级 LLM 并行方案，以优化数十万个片上核心的利用率。系统还提出 MeshGEMM 和 MeshGEMV，这是首批面向晶圆级加速器、能够有效扩展的 GEMM 与 GEMV 实现。

评测表明，与最先进方法相比，WaferLLM 的加速器利用率最高可提升 200 $\times$ 。在晶圆级加速器 Cerebras WSE-2 上，WaferLLM 的 GEMV 运算速度比 NVIDIA A100 GPU 快 606 $\times$ ，能效高 16 $\times$ 。对于完整 LLM 推理，WaferLLM 相比运行 SGLang 和 vLLM 的 A100 GPU 集群取得 10–20 $\times$ 加速。随着晶圆级 AI 模型、软件和硬件不断成熟，这些优势有望进一步扩大。WaferLLM 已在 <https://github.com/MeshInfra/WaferLLM> 开源。

1 引言

大语言模型 (LLM) 推理是一类快速增长的工作负载。它包含两个阶段 [15]：(i) 预填充阶段，处理输入 token（提示词），绝大多数周期用于通用矩阵乘法（GEMM）；(ii) 解码阶段，以自回归方式逐个生成 token，主要执行通用矩阵向量乘法（GEMV）。解码需要反复把整个 LLM 模型加载到片上内存，GEMV 占据了大部分周期。LLM 往往要生成大量 token，尤其是在测试时扩展场景中，例如 OpenAI-o1/o3 [16, 33] 和 DeepSeek-R1 [13]；因此推理受到 GEMV 延迟约束，在本质上受限于内存带宽。

为突破内存带宽瓶颈，AI 加速器正日益采用晶圆级系统集成 [21]。这种方法把芯片面积扩展到一整片晶圆，最大可达到典型 GPU 裸片的 100 $\times$ ，从而显著增加片上核心、内存与带宽。Cerebras WSE [24] 和即将推出的 Tesla

Dojo [41] 都是其中的例子。以 Cerebras WSE-2 为例，它集成了 850,000 个核心和 40GB 片上内存，片上内存容量是 GPU 的 1,000 $\times$ ，并提供 22PB/s 内存带宽，是 GPU 的 7,000 $\times$ 。台积电预计晶圆级系统集成将得到广泛采用，并指出其在性能、节能的裸片间通信和成本降低方面的优势。IEEE 同样预测，到 2027 年晶圆级计算将快速增长 [21]。晶圆级加速器已经投入真实部署，尤其用于模型服务。2025 年 2 月，Mixtral 和 Perplexity 采用晶圆级芯片，并在每美元 token 数上实现了与 GPU 相当的成本 [40, 44]。G42 目前运营着完全配备晶圆级加速器的数据中心，Cerebras 也已获得重要商业合同 [39]。

释放晶圆级加速器的潜力并不容易，因为现有 LLM 系统依赖 GPU 和 TPU 常见的共享内存架构。晶圆级加速器则采用片上网络 (NoC) 设计，把数百万个带本地内存的核心互连成一种超大规模、基于网格的内存架构。其规模远超片上交叉开关（例如 GraphCore IPU 的单跳 NUMA）、多插槽 NUMA [2]，以及高密度 AI 集群（每个 pod 中有数百块 GPU）[17]。若不能充分应对这一内存架构的根本转变，直接把 T10 [25] 和 Ladder [45] 等最先进系统的设计应用到晶圆级设备，往往会产生极差的性能。

为应对这些挑战，我们提出一种设备模型，用来刻画晶圆级加速器的关键硬件属性，并突出其与共享内存设备之间的重要差异。借助该模型，我们可以评估现有 LLM 推理设计原则，识别其中不符合模型的部分，并确定需要采用新方法的位置。在该模型指导下，我们得以实现一项富有挑战性的系统设计：在单颗芯片上运行完整 LLM 推理，尽量减少代价高昂的片外通信，并最大化片上内存带宽利用率。

上述思路促成了首个晶圆级 LLM 推理系统 WaferLLM，其贡献包括：

(1) **晶圆级加速器的设备模型**。我们提出 PLMR 模型¹，它刻画了晶圆级加速器的关键硬件属性：(i) 海量**并行核心 (P)**：一片大晶圆上可以集成数百万个核心，系统必须有效划分 LLM 及其运算；(ii) 高度非均匀的内存访问**延迟 (L)**：跨核心数据访问存在显著差异，延迟差距最高可达 1,000 $\times$ ，系统必须缓解这一问题；(iii) 受限的单核本地内

¹PLMR 可读作“Plummer”。

存 (M): 每个核心只有数十 KB 到数 MB 内存, 必须高效使用; (iv) 有限的硬件辅助路由 (R): NoC 路由硬件受单核面积限制, 只能支持有限条路由路径, 例如 Cerebras WSE-2 上少于 2^5 条。

(2) **晶圆级 LLM 并行**. 我们为晶圆级加速器提出一种有效且符合 PLMR 的 LLM 并行策略。在预填充阶段, 我们设计细粒度划分, 以实现百万核心并行。在解码阶段, 张量维度不足以继续划分, 因此我们设计细粒度复制, 以尽可能低的通信成本实现并行。由此, WaferLLM 获得比基于 GPU 的方法规模更大、粒度更细的并行性 (满足 PLMR 中的 P)。此外, 我们用面向 PLMR 模型的新算法替代传统的 GPU GEMM 和 GEMV 算子 (满足 L、M 和 R), 并提出无需矩阵转置的张量放置策略, 因为转置在网格 NoC 上代价很高 (满足 L)。

我们还为晶圆级设备设计了一种可扩展的 KV cache 管理方法。该方法采用新的 KV cache 移位机制, 确保各核心均衡使用 (满足 P 和 M), 避免 GPU 上常见的 KV cache 拼接方法造成核心利用率倾斜。

(3) **晶圆级 GEMM**. 我们提出可扩展的晶圆级 GEMM 算法 MeshGEMM, 用于加速预填充阶段。不同于传统分布式 GEMM 算法, MeshGEMM 通过两个关键操作完整满足 PLMR: 循环移位与交错排列。循环移位在保证算法正确性的同时, 把本地内存用量限制在有界范围内 (满足 M)。交错排列则尽量降低网格 NoC 上的通信延迟, 从而有效减轻高度非均匀内存延迟 (满足 L) 与路由资源 (满足 R) 的开销。

(4) **晶圆级 GEMV**. 我们提出可扩展的晶圆级 GEMV 算法 MeshGEMV, 用于加速解码阶段。与现有 GEMV 实现不同, MeshGEMV 使用新的 K 树 *allreduce* 算法, 在海量核心之间聚合本地 GEMV 结果。该算法使路由资源用量符合硬件限制 (满足 R), 并降低通信延迟 (满足 L)。

我们在 Cerebras WSE 引擎上实现了 WaferLLM: LLM 并行、MeshGEMM 和 MeshGEMV 约使用 7,000 行 CSL (一种类 C 编程语言), 加载 LLM checkpoint、启动推理和制定执行并行策略约使用 2,000 行 Python。

我们使用多种模型开展端到端 LLM 推理实验, 包括完整的 LLaMA3-8B 和 LLaMA2-13B, 以及 CodeLLaMA-34B 和 QWen2-72B 的若干层。结合晶圆级 LLM 并行、GEMM 和 GEMV 后, WaferLLM 优于最先进 (SOTA) 系统: (i) 比面向分布式片上内存架构海量核心的 SOTA 系统 T10 [25] 快 100–200 $\times$; (ii) 比面向共享内存架构的 SOTA 系统 Ladder [45] 快 200–400 $\times$ 。

微基准还表明, MeshGEMM 比 Cerebras WSE 默认优化 GEMM——SUMMA [42], 以及面向大规模网格

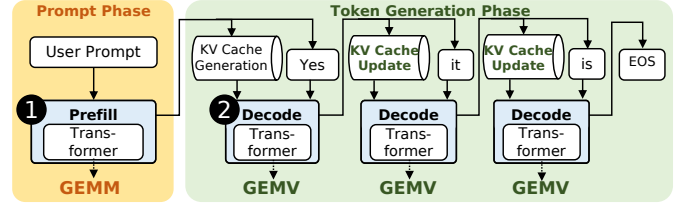


图 1: LLM 推理的关键组成部分

超级计算机的 SOTA GEMM——Cannon [6] 快 2–3 $\times$ 。MeshGEMV 比 Cerebras 优化的 GEMV 快 4–8 $\times$, 比单块 A100 GPU 快 606 $\times$ 。此外, WaferLLM 的 cache 移位方法比 GPU 上的 KV cache SOTA (例如 PagedAttention [20]) 最高可扩展 400 $\times$ 。

综合来看, Cerebras WSE-2 上的 WaferLLM 比单块 A100 上的 SGLang 快 30–40 $\times$ 。与通过 NVLink 和 RDMA 互连的多块 A100 上 SGLang 的最佳性能相比, WaferLLM 的端到端速度快 10–20 $\times$, 能效高 2.5 $\times$ 。从 GEMV 到完整 LLM, 收益有所缩小, 这是由当前软件、硬件和既有 LLM 模型设计的限制所致。我们预计, 随着晶圆级 AI 计算走向成熟并逐步解决这些限制, 其性能优势还会增强。

2 背景与动机

2.1 LLM 推理及其关键约束

如图 1 所示, LLM 推理系统通常以自回归方式逐 token 生成。模型由多个 Transformer 层组成, 其中主要是自注意力与前馈模块。推理分为预填充和解码两个阶段: 预填充阶段的总周期主要由 GEMM 运算主导 (①); 解码阶段的总周期则主要由 GEMV 运算主导 (②)。

LLM 推理受限于内存带宽。模型权重 (10–1,000GB) 在推理期间要反复从外部内存取入, 因为 GPU 通常只有约 100MB 片上内存。对于单个请求, 要达到每秒生成数千 token, 需要数百 TB/s 带宽, 远超当前 GPU 上 HBM (高带宽内存) 的能力。

尽管跨 GPU 的张量并行可以增加带宽, 但在大型 GPU 集群中缓解通信瓶颈依然困难。增加 GPU 还能提高并发查询的吞吐量, 却不能降低单个输出 token 的时间 (TPOT), 因为每个查询仍受内存带宽限制。

2.2 采用晶圆级加速器的原因

为了提高内存带宽, 加速器设计者出于以下原因正越来越多地采用晶圆级系统集成 [21]:

性能优势. 如表 1 所示, 晶圆级系统技术允许在单颗晶圆级芯片中集成数万亿晶体管, 最多是典型 GPU 裸片的

	裸片级系统	晶圆级系统
面积 (mm ²)	最高 858	最高 73062
晶体管数 (TSMC n3)	约 1 万亿	约数十万亿
核心数	数千-数万	十万-百万
片上内存	数十-数百 MB	约数十 GB
内存带宽	数 TB/s	约数十 PB/s
连接的 HBM	约数十-数百 GB	数十 TB (通过 TSMC SoW) [21]
裸片间带宽 (TB/s)	约 1-10 (片外)	约 10-100 (片上)
裸片间延迟 (ns)	约数百 (片外)	约个位数 (片上)
裸片间能耗 (pJ/bit)	约数十 (片外)	约零点几 (片上)

表 1: 裸片级系统与晶圆级系统的比较

100 $\times$ 。因此可以集成数百万个面向 AI 优化的核心，提供数十 GB 片上内存和最高数十 PB/s 内存带宽，后者是标准 GPU 数 TB/s 带宽的 1,000 $\times$ 。与标准裸片相比，未来的晶圆级芯片还可在边缘连接多 40-80 $\times$ 的 HBM 芯片 [21]。

集成效率。 晶圆级系统擅长集成海量并行核心。与传统基于 PCB 的芯片间互连（例如 NVIDIA NVLink、PCIe）相比，晶圆上的裸片间连接可在单位面积上提供最高 10 $\times$ 带宽、100-300 $\times$ 的延迟优势，以及每 bit 近 100 $\times$ 的能效提升 [24]，如表 1 所示。前文已经指出，LLM 推理因密集数据访问而主要受内存带宽约束。在分布式环境中，远程内存访问引起的芯片间通信开销会限制扩展性，解码阶段的 GEMV 尤其如此。因此，与传统芯片间链路相比，晶圆级系统集成能提供延迟更低、带宽更高的互连，并改善效率。

更低成本。 晶圆级集成可以降低制造成本，因为制造成本中相当大的一部分（通常为 30-50%）来自对单颗芯片的测试与封装 [48]。此外，晶圆级集成在提升良率方面已取得显著进展。台积电等公司也在开发把经过完整测试的裸片集成到同一晶圆上的技术，以进一步改善良率。

2.3 晶圆级 LLM 推理面临的挑战

利用晶圆级加速器进行 LLM 推理的核心挑战，在于它们在单颗芯片上转向了分布式、非均匀内存架构。如图 2(a) 所示，现有 LLM 系统针对共享内存（单芯片）或全互连架构（例如 GPU pod）进行了优化。然而，随着片上内存容量增长，这些架构面临指数增长的制造成本和性能下降，因而必须采用分布式片上架构。

如图 2(b) 所示，AI 加速器设计者主要使用**类网格片上网络 (NoC)** 连接**海量核心**（从数十万到数百万）。网格拓扑有利于高效排列核心，并能有效解决散热 [29]、供电 [19] 和低成本布线 [38, 46] 问题；每个核心只与附近邻居通信，如图 2(b) 所示。3D 环面或树结构等其他拓扑因片上布线成本过高而不切实际。因此，Cerebras WSE [24] 和 Tesla Dojo [41] 等晶圆级芯片厂商均采用超大规模网格

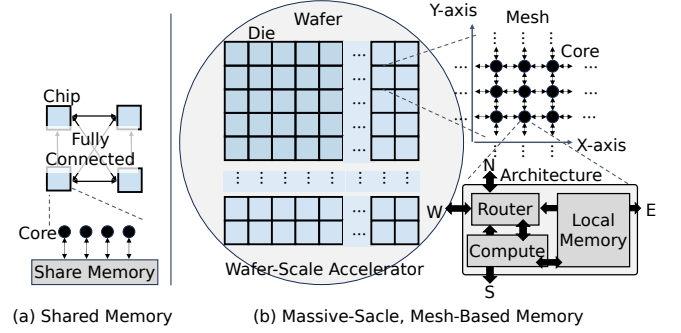


图 2: 超大规模、基于网格的内存架构

架构。即使是 Meta MTIA [30]、Tenstorrent [18] 等非晶圆级加速器 [4, 31]，也使用网格来扩展单芯片内的核心数量。

超大规模网格架构给多种 LLM 运算带来挑战，因为它们需要大量数据移动：(i) 管理 LLM 模型与 KV cache；(ii) 预填充阶段的 GEMM 运算；(iii) 解码阶段的 GEMV 运算。点积和激活函数等逐元素运算不需要移动数据，天然受益于并行。RMSNorm 和 Softmax 等需要 allreduce 的运算，则可以复用 GEMV 方案。

3 晶圆级加速器设备模型

3.1 PLMR 模型

我们提出 PLMR 模型，用于刻画晶圆级加速器独特的硬件属性，并由此推导利用这类新兴硬件所需满足的系统要求。

- (1) **海量并行性 (P)**：晶圆级加速器可以容纳数百万个并行核心，而 GPU 通常只有数千个。每个核心都带有本地硬件流水线，可在周期粒度上重叠数据输入、数据输出、计算与内存访问。因此，计算必须进行超大规模划分，并通过细粒度调度来重叠计算、内存访问和 NoC 通信。
- (2) **高度非均匀的内存访问延迟 (L)**：在网格中跨核心访问内存会产生高度非均匀的延迟。对于一个按矩形排列、包含 $N_w \times N_h$ 个核心的网格，任意两核之间的最大 NoC 跳数为 $N_w + N_h$ ，最坏情况下的内存访问延迟为 $\alpha(N_w + N_h) + \beta r$ ，其中 $r < N_w + N_h$ 表示通信路径上的路由阶段数。这里， α 是单跳传输延迟，即消息在某个核心依据路由硬件预配置规则被直接转发时产生的成本，它随跳数增加； β 是单次路由延迟，即消息在某个核心转发时由软件解析并重写报头所产生的开销 [27]。通常 $\alpha < \beta$ 。在拥有一百万个核心的网格中，最大跳数与路由次数可达数千，从而使本地与远程内存访问的延迟相差最高一千倍。因此，尽量减少长距

离通信对性能至关重要。

- (3) **受限的单核本地内存 (M)**：每个核心只有很小的本地内存（数十 KB 到数 MB），因为更大容量会降低性能与能效 [47]。因此，必须把计算数据显式划分为细粒度块，使其完整容纳于每个核心的本地内存限制之内。
- (4) **受限的路由资源 (R)**：集成数百万核心的晶圆级加速器会严格限制每个核心的路由电路复杂度或路由表大小。例如在 Cerebras WSE-2 上，每个核心只能识别带 5-bit 地址编码的消息报头。因此，每个核心最多支持 2^5 条不同路由路径，软件系统必须仔细规划这些路径。对于长距离远程通信，一对核心可以消耗路由资源建立直接路由路径，只产生 α 延迟；但如果路由路径数超过硬件限制，消息就必须经由多个中间核心中继，从而引入额外的 β 延迟。

我们预计这些属性会持续成立，因为它们源自硬件及其制造工艺的基本特性。PLMR 模型既适用于当前的 Cerebras WSE，也适用于未来的 Tesla Dojo 等晶圆级设备。甚至一些采用网格 NoC、但并非晶圆级的设备，例如 Tenstorrent Blackhole [18]，也可以用 PLMR 表示，只需调整并行度 (P)、网格规模 (L) 等参数，或放宽本地内存 (M) 和路由资源 (R) 的约束。

3.2 最先进方法的局限

借助 PLMR 模型，我们分析现有 AI 系统为何无法充分利用晶圆级加速器。要在晶圆级加速器上运行 LLM，通常有两种选择：(i) 把每个核心的分布式本地内存抽象成共享内存，并通过 NoC 直接访问放置在远程核心上的数据；(ii) 把计算显式划分到分布式核心，并用消息传递交换必要数据。我们分析两类有代表性的系统：一类是面向 GPU 等共享内存架构的 LLM runtime 或 DNN 编译器，例如 Ladder [45]；另一类是面向分布式片上内存架构的 SOTA 编译器，例如用于 GraphCore IPU 的 T10 [25]。

共享内存系统。 Ladder 等基于共享内存的 DNN 编译器通常假设底层内存层次中的访问模式是均匀的，因而无法容忍晶圆级加速器访问远程内存时高达数千倍的延迟差异（不满足 L）。此外，这些编译器 [10, 28, 37, 45, 51, 54, 57] 常把重点放在计算划分上，而较少优化数据划分。这很容易造成大量数据重复，并违反内存约束要求（不满足 M）。最后，这类编译器不知道各核心之间的通信距离，因而也无法妥善处理路由资源约束。

分布式内存系统。 T10 系统 [25] 面向带片上交叉开关的 AI 加速器设计；这种交叉开关保证访问同一芯片上任意其他核心内存的延迟恒定。T10 能处理小容量本地内存并平衡通信负载，因此满足内存约束 (M) 和路由资源限制 (R)。但在 PLMR 设备上，它没有考虑变化的内存访问延

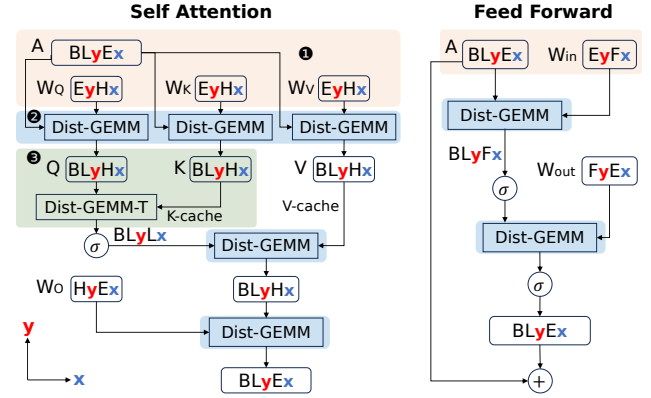


图 3: 预填充并行方案。 $E_x F_y$ 表示形状为 EF 的矩阵，其中 E 维沿网格核心的 x 轴划分， F 维沿 y 轴划分。

迟（不满足 L），并且只能扩展到数千而非数百万核心（不满足 P）。

4 晶圆级 LLM 并行

本节介绍晶圆级 LLM 并行方案，其中针对预填充、解码和 KV cache 管理都采用了新的设计。

4.1 预填充并行

LLM 预填充的并行方案必须符合 PLMR 模型。主要挑战包括：(i) 预填充期间要处理多个大矩阵，因此需要有效划分维度以实现百万核心并行 (P)；(ii) 优化 GEMM 运算还涉及进一步划分，以及计算与通信的重叠，既要尽量降低长距离通信延迟 (L)，又要遵守本地内存约束 (M) 并考虑有限的路由资源 (R)；(iii) 连续 GEMM 运算经常需要矩阵转置，而转置在网格 NoC 上代价很高 (L)。

为百万核心并行设计细粒度划分。 为获得较高芯片利用率，我们建议把输入激活矩阵和权重矩阵的两个维度同时沿核心的 X 轴与 Y 轴划分。与通常只划分嵌入维度的现有方法 [12, 15, 32, 35] 相比，这种方法能够实现粒度更细的百万级并行；现有方法在 PLMR 设备上的并行度不足。

我们用自注意力和前馈模块说明这种划分，如图 3 所示。为便于讨论，定义以下记号：预填充过程中，输入激活 A 与权重 W 都是多维张量。 B 表示 batch size， L 表示序列维度²， E 表示嵌入维度， H 表示注意力头维度， F 表示前馈模块的隐藏维度。如图中的 ① 所示， A 的划分局部记作 $BL_y E_x$ ，其中 L 维沿核心的 Y 轴划分， E 维沿 X 轴划分。所有权重矩阵 (W_Q 、 W_K 、 W_V 、 W_{in} 和 W_{out}) 也同样沿两个维度划分。

设计符合 PLMR 的分布式 GEMM。 如图 3 中的 ② 所示，我们建议在预填充阶段，以新设计的、符合 PLMR 的分布式 GEMM 替代面向共享内存架构的传统 GEMM 算

² L ：序列维度； L ：PLMR 中的内存访问延迟。

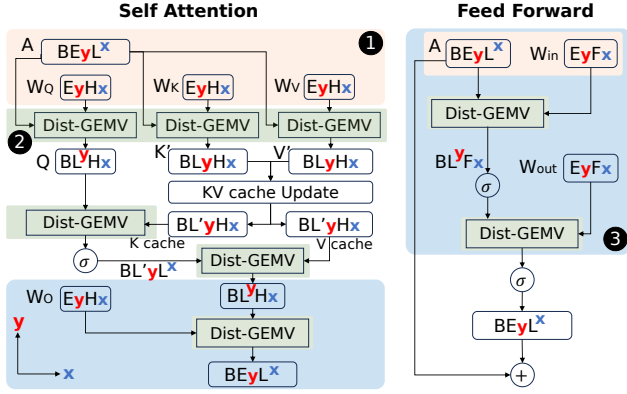


图 4: 解码并行方案。 $E^y F_x$ 表示 E 维沿 y 轴复制, F 维沿 x 轴划分。

子。TPU 和 GPU 系统的 GEMM 主要依赖 allgather, 而符合 PLMR 的分布式 GEMM 算法在遵守本地内存和路由约束的同时实现较高 NoC 带宽利用率, 因而满足 L、M 和 R 属性。第 5 节将完整介绍该分布式 GEMM。

使用转置式分布式 GEMM 避免矩阵转置。我们为预填充提出一种免转置并行方案, 以避免共享内存 LLM 系统中的常见操作——矩阵转置。PLMR 的 L 属性指出, 矩阵转置在晶圆级设备上尤其昂贵: 它可能要求网格一个角上的核心把数据发送到对角的另一角, 从而形成长距离通信路径。

如图 3 中的 ③ 所示, 我们的免转置并行方案使用转置式分布式 GEMM (记作 dist-GEMM-T) [11, 42], 计算 LLM 预填充中的 $Q @ K^T$ 。具体而言, Q 和 K 中间张量分别由 X 与 W_Q 、 W_K 相乘得到; 受片上划分形状影响, 若采用普通 dist-GEMM, 必须先转置 K , 而 dist-GEMM-T 可以免去这一步。

4.2 解码并行

LLM 解码是内存带宽密集型阶段, 其并行策略面临以下挑战: (i) 受输入序列和 batch size 限制, 解码使用的矩阵小于预填充, 因此当维度不足以划分时必须谨慎并行; (ii) 该阶段高度依赖 GEMV。GEMV 的计算强度低于 GEMM, 计算阶段较短, 难以充分与通信重叠, 因此容易受到网格 NoC 长距离通信延迟 (L) 的影响, 同时还要遵守本地内存和路由约束 (M 和 R); (iii) 连续 GEMV 运算会引入 NoC 上代价高昂的矩阵转置, 可能违反 L 属性。

设计细粒度复制, 以最低通信成本实现并行。当张量维度不足以达到解码所需的高并行度时, 我们建议对 LLM 张量进行细粒度复制, 尤其是复制序列维度: 预填充阶段的序列长度等于提示词长度, 而解码阶段等于 1。这种方法有两项主要优势: (i) 提高并行度, 确保所有核心的负载均衡; (ii) 避免跨全部核心的 allreduce 等额外通信。如

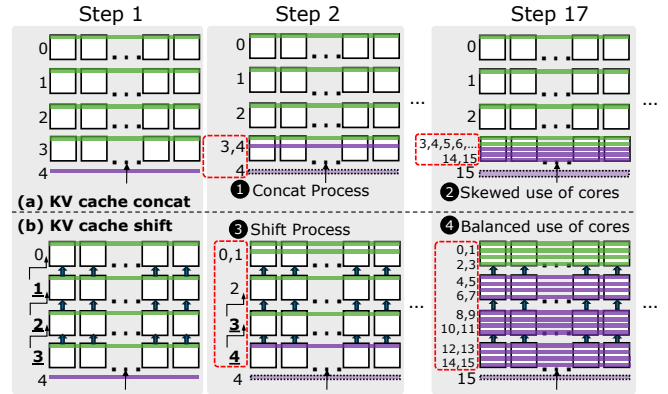


图 5: KV cache 拼接与 KV cache 移位的比较

图 4 中的 ① 所示, E 维沿 y 轴划分, L 维沿 x 轴复制, 记作 $BE_y L^x$ 。权重矩阵 W 仍沿两个维度划分, 与预填充阶段一致。

我们的细粒度复制不同于近期的长上下文/序列推理系统 [49, 55]; 这些工作只在预填充而非解码阶段选择性地复制某些维度。

设计符合 PLMR 的分布式 GEMV。我们发现, 现有 GEMV 实现因长距离通信和单核路由资源消耗过多而无法完整满足 PLMR。为此, 我们提出符合 PLMR 的分布式 GEMV, 并在整个解码阶段使用这一新实现, 如图 4 中的 ② 所示。第 6 节将完整介绍其设计。

预先优化模型权重放置以避免矩阵转置。为避免解码期间的矩阵转置, 我们针对解码, 尤其是分布式 GEMV, 预先优化模型权重布局。这样便可消除矩阵转置。尽管预填充与解码之间会产生重新放置的开销, 但它远小于 token 生成期间连续执行矩阵转置的开销。

图 4 的 ③ 展示了这一方案。具体来说, 我们为解码时的分布式 GEMV 优化 W_O 、 W_{out} 等权重的放置方式, 使其不同于预填充阶段的布局。该方法还消除了了解码自注意力中计算 $Q @ K^T$ 时的转置操作。

4.3 基于移位的 KV cache 管理

在 PLMR 设备上管理 KV cache 很有挑战性: 一方面, 大量数据必须分布存储在多个核心上并遵守本地内存约束 (M); 另一方面, KV cache 计算必须分散到多个核心以实现高并行度 (P)。对此, 我们得到以下认识。

现有的拼接式管理造成核心利用率倾斜。当前 KV cache 管理方法主要把最新生成的 KV 向量拼接 to 既有 cache 末尾。这一操作在共享内存架构上很高效, 但在 PLMR 设备上会造成核心利用率严重倾斜。如图 5 的 ① 所示, 每一行只有一个核心负责存储生成的 KV 向量并对其进行计算。生成若干 token 后, 这个唯一的核​​心很快就会成为瓶颈, 如图 5 的 ② 所示; 这造成内存使用倾斜并违反 PLMR 中

的 M。与此同时，KV cache 在核心间分布不均也会降低并行效率，违反 P 属性。

提出基于移位的管理以均衡核心利用率。 我们提出基于移位的 KV cache 管理策略，把 cache 数据均匀分布到所有核心。该方法不把新的 KV cache 向量拼接到末尾，而是执行负载均衡移位：每一行把最旧的 KV cache 数据传给上一行，如图 5 的 ③ 所示。新 KV 数据到达时，每个核心都会把本地容量与邻居进行比较；若容量相等，便触发向上移位，每一行从下一行接收数据，并把一部分数据传给上一行。如 ④ 所示，这能保证 KV cache 均匀分布在所有核心上。

向上移位并行使用 NoC 链路，因此保持较高性能并满足 P 属性。KV cache 的物理放置与逻辑连续性一致，而且只在相邻核心之间移动数据，符合 L 属性。该方法还彻底解决了拼接式方案中最后一行核心违反 M 约束的问题。

4.4 实现细节

下面概述若干实现细节。

预填充与解码的转换。 预填充和解码需要不同策略。为高效完成转换，我们通过高速 NoC 重新排列 KV cache 和权重。NoC 的聚合带宽通常达到数百 Pbit/s，因此可以即时完成，而无需依赖较慢的片外内存。

并行配置。 WaferLLM 使用离线自动调优，为每个模型选择核心数量，并依据模型大小、输入/输出长度、单核内存以及预填充/解码阶段来优化延迟。系统会为两个阶段选择不同的核心数，而高带宽 NoC 支持快速动态重映射。对于输入/输出长度可变的模型，系统采用平均值以保持接近峰值的性能。每个模型都单独执行自动调优，以适配其具体需求。

自注意力变体。 WaferLLM 支持分组查询注意力 [3]、多头注意力 [43] 和多查询注意力 [5] 等自注意力变体。它们先按注意力头维度分组，然后在组内执行 dist-GEMM、dist-GEMV 和 dist-GEMM-T。

5 晶圆级 GEMM

本节介绍 MeshGEMM：一种可扩展到超大规模网络架构的分布式 GEMM。

5.1 分布式 GEMM 的 PLMR 符合性

为了找出适用于 PLMR 设备、能够扩展的分布式 GEMM，我们定义以下指标：(i) 单核路由路径数：每个核心使用的路由路径数量；路径越少，越符合 R 属性。(ii) 关键路径延迟：每一步传输子矩阵的所有通信路径中的最大延迟，即图 6 中的红线；延迟越低，越符合 L 属性。(iii)

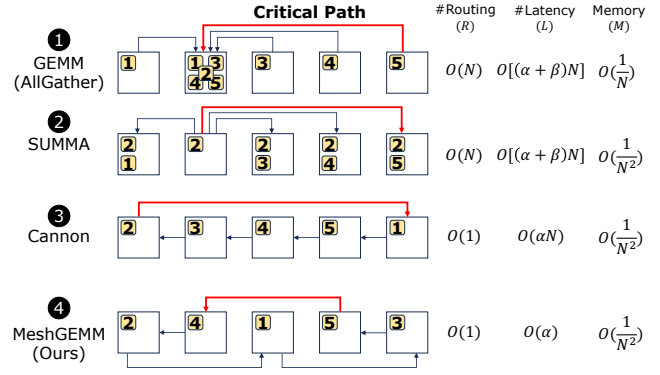


图 6: 分布式 GEMM 的 PLMR 符合性

单核内存：每个核心所需内存；用量越低，越符合 M 属性。

下面分析现有分布式 GEMM 方法，并说明 MeshGEMM 如何满足这些指标。

- (1) **基于 Allgather 的 GEMM** 常用于 GPU 和 TPU pod 上的分布式 GEMM [32, 35, 56]。每一步的关键通信路径，是某个核心从最远核心收集数据的路径，如图 6 的 ① 中红线所示；完成 allgather 共需 N 步。每个核心要为同一行和同一列中的邻居建立 N 条路由路径（违反 R）。R 约束迫使系统经由中间核心逐步传输子矩阵，使关键路径产生 $O[(\alpha + \beta)N]$ 的通信延迟（违反 L）；膨胀的工作缓冲区还使单核使用 $O(1/N)$ 内存（违反 M）。
- (2) **SUMMA** 是 Cerebras 晶圆级引擎默认采用的分布式 GEMM [7]。每一步的关键通信路径，是一个核心沿列或行把数据广播到最远核心的路径，如图 6 的 ② 中红线所示。与 ① 中的 allgather 相同，每个核心建立 N 条路由路径（违反 R），每一步关键路径的延迟为 $O[(\alpha + \beta)N]$ （违反 L）。尽管 SUMMA 的内存用量优于 allgather，只需要与本地划分子矩阵等大的工作集，但峰值内存仍会翻倍。
- (3) **Cannon** 是针对网络优化的分布式 GEMM [6]，广泛用于超级计算机与分布式集群。每一步的关键通信路径，是头部核心把数据发送到尾部核心的路径，如图 6 的 ③ 所示。每个中间核心在 2D 环面中与两个邻居通信，并将子矩阵从头部传递到尾部，因此只需 $O(1)$ 条通信路径，内存用量也达到最优的 $O(1/N^2)$ 。值得注意的是，Cannon 的单核路由路径数为常数，因此可以为邻居通信与关键路径分配静态路由规则。每一步中，子矩阵都可通过中间核心从头部直接传到尾部，不像基于 allgather 的 GEMM 或 SUMMA 那样必须逐步传递消息。因此，关键路径只产生单跳延迟 α ，没有额外的单次路由开销 β 。但是关键路径仍跨越 N

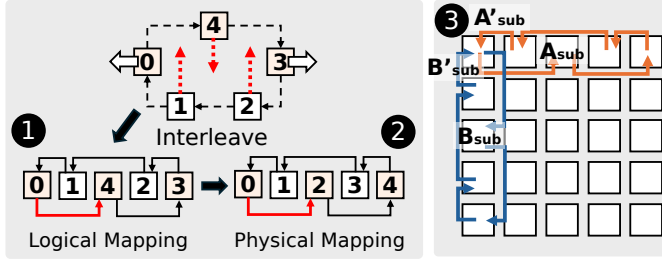


图 7: 设计直觉与可扩展性分析。

跳，所以延迟依然为 $O(\alpha N)$ ，如 ③ 的红线所示，违反 L。

- (4) **MeshGEMM (本文方法)** 是一种符合 PLMR 模型的分布式 GEMM。我们把每一步的关键通信路径称为“两跳传输”，如图 6 的 ④ 中红线所示；它显著短于其他方法。每个核心与两个相距两跳的邻居通信，并穿过一个单跳邻居的通信路径（后文将证明，该方法可以扩展到更大的网格架构）。与 Cannon 相同，这一设计只需要 $O(1)$ 条单核通信路径，内存用量也达到最优的 $O(1/N^2)$ 。关键在于，它把关键路径恒定限制为 2 跳，复杂度为 $O(\alpha)$ ，因而是这些方法中唯一能满足 L 属性的方案。

5.2 设计直觉与可扩展性分析

我们的设计分为两个阶段：(i) 通过 GEMM 的循环移位过程保证算法正确性；(ii) 再用交错排列把关键路径延迟限制为常数。循环移位与交错排列共同形成**两跳传输**通信路径；我们还证明，在这一循环上，两跳是满足 L 属性所需的最短距离。

循环移位。 循环移位把通信限制在两个邻居之间，并尽量降低内存用量，使 MeshGEMM 满足 M 与 R 属性。它以类似 Cannon [6] 的推理保证 GEMM 结果正确。如图 6 的 ③ 所示，一个由 5 个核心构成的逻辑环被展开到物理通信映射中，其中存在一条从头部核心到尾部核心的关键路径。

交错排列。 对于展开后的通信方案，我们希望进一步缩短关键路径，从而满足 L 属性。核心思路是引入 INTERLEAVE 操作，寻找从逻辑位置到物理位置的映射关系，其定义见算法 1。如图 7 的 ① 所示，MeshGEMM 先把核心 1 插入核心 0 与核心 4 之间，再把核心 2 插入核心 4 与核心 3 之间，形成逻辑映射；随后调用 INTERLEAVE，得到发送目标邻居与接收来源邻居的索引，最终得到图 7 中 ② 所示的、经过置换但等价的通信方案。例如，总共有 5 个核心 ($N=5$)，物理核心 2 (index=2) 便向物理核心 4 (send_index=4) 发送数据，并从物理核心

算法 1: INTERLEAVE

输入: index, N
输出: send_index, recv_index

```

1 若 index mod 2 == 0 则
2   |   recv_index = Max(index - 2, 0);
3   |   send_index = Min(index + 2, N - 1);
4 否则
5   |   recv_index = Min(index + 2, N - 1);
6   |   send_index = Max(index - 2, 0);
7 若 index == 0 则 recv_index = 1;
8 若 index == N - 1 则
9   |   若 N mod 2 == 0 则 recv_index = N - 2;
10  |   否则 send_index = N - 2;
11 返回 send_index, recv_index;

```

0 (recv_index=0) 接收数据。图 6 与图 7 以 5 个核心为例，而算法 1 表明，MeshGEMM 的两跳通信模式可以推广到任意 $N \geq 3$ 的网格架构。

可扩展性分析。 我们可以证明，INTERLEAVE 产生的两跳距离无法进一步缩短。证明依赖序列排列的基本性质：若试图构造一个循环序列，并要求每个数字与相邻数字恰好相差一跳，就会遇到数学上的不可能。把数字想象成直线上的点即可理解：相邻数字虽然可以连接，但序列的两个端点不可能在组成环的同时，仍与各自邻居都保持一跳之差。

以上讨论基于一维数组，但可以自然推广到二维网格，因为一维数组分别对应网格中对称的 X 轴和 Y 轴。

5.3 MeshGEMM 算法

MeshGEMM 的关键步骤如下。

- (1) **初始化：**考虑 $C = A \times B$ 。MeshGEMM 沿两个维度把 A 与 B 划分为 A_{sub} 和 B_{sub} tile，形成分布在各核心上的 $N \times N$ 个 tile。每个核心接收一个 A_{sub} tile 和一个 B_{sub} tile。随后，MeshGEMM 使用 INTERLEAVE 初始化每个核心的邻居位置。
- (2) **对齐：**每个核心与邻居对齐输入子矩阵，保证分布式系统中的每个核心在开始矩阵乘法时都持有正确的操作数。
- (3) **计算-移位循环：**每个核心执行一个包含 N 步通信与计算的计算-移位循环。每一步中，每个核心都计算其对应的部分和 $C_{sub} = A_{sub} \times B_{sub} + C_{sub}$ ；同时沿 X 轴移位 A_{sub} 、沿 Y 轴移位 B_{sub} ，得到下一步计算所需的新 A'_{sub} 与 B'_{sub} ，如图 7 的 ③ 所示。执行 N 步

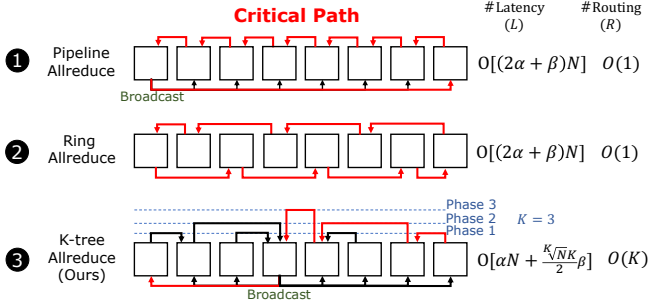


图 8: 分布式 GEMV 的 PLMR 符合性

后, 返回累积的 C_{sub} 。

5.4 实现细节

处理非方形网格. 对于非方形网格 $N_h \times N_w$ ($N_h \neq N_w$), 可以在逻辑上把 A 、 B 矩阵划分到 $N_{lcm} \times N_{lcm}$ 个核心, 其中 N_{lcm} 是 N_h 与 N_w 的最小公倍数。

转置式分布式 GEMM. 上述算法还可用于计算 $C = A \times B^T$, 即图 3 中用来避免在网格上转置 B 的 dist-GEMM-T。计算前无需对齐, 只需沿 Y 轴对右矩阵 B 执行 N 步两跳计算-移位。每一步移位后, 每个核心计算 $C_{sub} = A_{sub} \times B_{sub}$, 随后沿 X 轴对 C_{sub} 执行 ReduceAdd。经过 N 步即可得到最终矩阵 C 。

6 晶圆级 GEMV

本节介绍 MeshGEMV: 一种面向 PLMR 设备的可扩展 GEMV 算法。

6.1 分布式 GEMV 的 PLMR 符合性

分布式 GEMV 的完成时间主要由两部分决定: 首先, 每个核心执行逐元素计算, 生成局部分和; 随后, allreduce 聚合所有选中核心的部分结果, 并把聚合结果发回所有核心, 以便继续执行 GEMV。与 GEMM 的分析类似, 我们也为晶圆级 GEMV 定义以下指标: (i) 单核路由路径数: 每个核心使用的路由路径数量; 路径越少, 越符合 R 属性。(ii) 关键路径延迟: 整个 allreduce 通信的最大延迟, 主要由从最远核心发送部分和, 到该核心收到聚合和之间的时间决定。

下面分析现有分布式 GEMV 方法, 并说明 MeshGEMV 如何满足这些指标。

- (1) **采用流水线 allreduce 的 GEMV** 常用于 TPU pod 系统 [35], 也是 Cerebras 示例中的默认方案 [9]。如图 8 的 ① 中红线所示, 流水线 allreduce 的关键路径从最远核心开始, 部分和逐步向根核心归约, 随后再从根核

心把聚合结果广播到所有核心。尽管该方法把单核路由路径数控制在 R 约束之内, 即单核路由资源用量为 $O(1)$, 但它总共需要 $2N$ 跳和 N 个路由阶段, 违反 L 属性。

- (2) **采用环形 allreduce 的 GEMV** 常用于 GPU pod, 尤其在数据量较大时是默认配置 [27]。如图 8 的 ② 所示, 环形 allreduce 的关键路径要求每个部分和遍历环上的所有核心。与流水线 allreduce 相似, 该方法保持单核 $O(1)$ 条路由路径, 但通信延迟为 $O[(2\alpha + \beta)N]$, 因而违反 PLMR 定义的延迟约束 L 。
- (3) **采用 K 树 allreduce 的 GEMV (本文方法)**。如上所述, 流水线与环形 allreduce 顺序执行 reduce-add, 导致关键路径上存在 N 个路由阶段。与之不同, K 树 allreduce 把 reduce-add 路径组织成一棵平衡 K 树, 以 K 个阶段执行分组并行归约, 每组包含 $O(\sqrt[K]{N})$ 个核心, 从而把关键路径减少到仅 $\frac{\sqrt[K]{N}K}{2}$ 次路由和 N 跳; 代价是每个核心需要 $O(K)$ 条路由路径。

6.2 MeshGEMV 算法

MeshGEMV 的关键步骤如下。

- (1) **初始化:** 考虑 $C = A \times B$, 其中 A 为向量。MeshGEMV 沿两个维度把 B 划分为 B_{sub} tile, 形成分布在各核心上的 $N \times N$ 个 tile。对于 A , MeshGEMV 沿向量长度把它划分成 N 个 tile, 在一个轴上分布, 并在另一轴上复制。每个核心接收一个 A_{sub} tile 和一个 B_{sub} tile。随后依据 K 树确定每个阶段中哪些核心组成一组, 以获得聚合结果。
- (2) **并行计算:** 每个核心执行本地 GEMV, 即 $A_{sub} \times B_{sub}$, 得到部分和 C_{sub} 。
- (3) **聚合:** 聚合主要采用我们设计的 K 树 allreduce, 步骤如下: (i) 在第 1 阶段, 每个组执行组内归约, 并在各组根核心上得到 C_{sub} 的部分和; (ii) 在第 k 阶段, 把第 $(k-1)$ 阶段的结果归约到第 k 阶段各组的根核心。重复 K 次后, 拼接所有 K 树根核心上的 C_{sub} 即可得到 C ; (iii) 是否继续从 K 树根核心执行一次可选的广播, 取决于后续是否需要连续 GEMV。

可扩展性分析. 如图 8 的 ③ 所示, 通过选择合适的 K , 该方法能够随并行规模高效扩展并满足 L 属性。树根核心需要 $K+1$ 条路径, 但可以根据硬件的 R 限制灵活调整 K 。与流水线和环形 allreduce 相比, K 树 allreduce 本质上是消耗路由资源, 在远距离核心之间建立直通路, 从而减少路由延迟。

不过, 更大的 K 并不总是更好, 因为它取决于 N 和 R 约束; 同时, 增大 K 还会提高路由复杂度与开销。综合这些因素, 我们在当前实现中选择 $K=2$, 并在后续章

节中进行评测。

7 评测

我们将 WaferLLM 与多种最先进方法和系统进行了广泛比较。结果表明：

- (1) 在 LLM 推理中，WaferLLM 相比 T10 和 Ladder 获得了数量级的加速（第 7.1 节）；
- (2) WaferLLM 的 MeshGEMM 和 MeshGEMV 在性能与强扩展性上都优于最先进方法（第 7.2 节）；
- (3) WaferLLM 基于移位的 KV cache 管理把 token 容量提高了 $360\text{--}385\times$ 以上（第 7.4 节）；
- (4) 与 A100 GPU 集群上 SGLang 的最佳性能相比，Cerebras WSE-2 上的 WaferLLM 取得 $10\text{--}20\times$ 端到端加速；与单块 A100 GPU 相比，加速约为 $30\text{--}40\times$ 。WaferLLM 的能效也高出约 $2.5\times$ （第 7.1、7.5 节）。

实验设置. 我们在一台配备 Cerebras WSE-2 的服务器上评测 WaferLLM。WSE-2 拥有 850,000 个核心，每个核心都带有最高工作频率为 1.1GHz 的计算引擎（CE）。每个时钟周期可以从 SRAM 取入两个 32-bit 操作数，执行一次乘加运算，再把结果写回 SRAM。每个核心还有一个 fabric router，可在单个时钟周期内向邻近核心发送或从其接收 32-bit 消息。此外，每个核心包含 48KB SRAM，全芯片聚合 SRAM 容量为 40GB [24]。

我们把 WaferLLM 与两种 DNN 编译器进行比较：(i) T10 [25]，面向核心间互连和分布式片上内存 AI 加速器的最先进编译器；(ii) Ladder [45]，面向共享内存架构的最先进编译器。对于 T10，我们在 WSE-2 上实现该系统，并把每个核心视为由交叉开关互连的分布式内存系统的一部分，尽管实际拓扑是网格。T10 把数据映射到核心 ID，并按需从本地 SRAM 取入数据。对于 Ladder，我们把芯片上由网格互连的分布式内存架构视作统一内存，因此访问数据需要在 NoC 上执行集合通信。

GPU 对比实验使用 NVIDIA A100；为保证公平，它与 WSE-2 均采用 7nm 工艺。实验最多使用跨两个节点的 16 块 GPU (2×8 配置)。单节点内的 8 块 A100 通过 NVLink 互连，节点间通信使用高性能 InfiniBand (IB) 网络。GPU 实验采用性能最高的 LLM 推理系统之一 SGLang [53]。

评测指标. 为评估不同硬件上 LLM 推理系统的单请求最大吞吐性能，我们定义每请求吞吐量（Throughput per Request, TPR）这一关键指标。TPR 由更常用的每输出 token 时间（TPOT）推导而来： $\text{TPR} = \frac{1}{\text{TPOT}}$ 。

LLM 模型. 评测覆盖了不同规模与架构的多种代表性 LLM。LLaMA3-8B 和 LLaMA2-13B 是广泛使用的开源 LLM；其中 LLaMA3 使用分组查询注意力而非多头注意

模型	设备		输入/输出序列长度			
			2048/128	4096/128	2048/2048	4096/4096
LLaMA3-8B	WSE-2	WaferLLM	764.4	604.4	2370.3	2459.0
		T10	4.6	4.5	58.3	94.6
		Ladder	1.2	1.1	7.4	8.7
	A100 GPU (SGLang)	1	34.8	31.1	36.5	78.4
		8	117.2	109.0	128.4	256.1
LLaMA2-13B	WSE-2	1	73.7	70.2	79.3	162.5
		8	240.4	217.1	240.4	479.9
		2×8	79.6	70.5	86.9	172.4
	A100 GPU (SGLang) ³	1	20.4	17.1	21.1	47.9
		8	79.6	70.5	86.9	172.4

表 2: 端到端 LLM 推理的 TPR

力，以减少 KV cache 用量。对于 LLaMA2-13B，我们移除了模型的 4K 上下文长度限制，以评估系统在不同输入、输出长度下的性能。此外，受张量并行约束，模型的注意力头数必须能被所用 GPU 数整除，因此我们没有在 16-GPU 集群上开展 LLaMA2 的多 GPU 实验。CodeLLaMA-34B 是面向编程任务的专用 LLM，QWen2-72B 则是另一种以高模型质量著称的流行 LLM。

7.1 LLM 推理

我们首先报告 WaferLLM 与 WSE-2 上 T10、Ladder 以及 A100 GPU 集群上 SGLang 的端到端性能比较；随后把执行拆分为预填充和解码阶段，以进一步分析性能并提供更深入的认识。

端到端吞吐量. 表 2 给出了不同输入、输出序列长度下，LLaMA3-8B 和 LLaMA2-13B 在 WSE-2 与 A100 上的推理 TPR。这里，端到端吞吐量等于解码阶段生成的 token 总数除以预填充和解码两个阶段的总时间。WaferLLM 针对每个模型使用经过优化、能获得最佳性能的核心配置：LLaMA3-8B 的预填充使用 660×660 个核心，解码使用 360×360 个核心；LLaMA2-13B 的预填充使用 750×750 个核心，解码使用 375×375 个核心。由于单颗 WSE-2 的内存容量不足，表中未包含 CodeLLaMA-34B 和 QWen2-72B。

与 T10 相比，对于 4096 或 2048 输入上下文、输出 128 个 token 的短序列生成任务，WaferLLM 平均加速 $160\times$ ，最高 $180\times$ ；对于输入上下文分别为 4096、2048 token，输出长度分别为 4096、2048 token 的较长任务，平均加速 $36\times$ ，最高 $48\times$ 。T10 的 compute-shift 模型虽然考虑了 PLMR 设备的内存约束 (M) 和路由资源限制 (R)，却没有考虑核心由网格 NoC 互连这一事实，因此既无法处理不同跳数带来的差异 (L)，也无法扩展到数百万核心 (P)。这说明，超大规模 NUMA 架构需要新的系统设计。

与 Ladder 相比，对于上述输出 128 个 token 的短序

³受模型架构和张量并行约束，没有 2×8 GPU 结果。

模型	方法	WSE-2 核心			A100 GPU 数 (SGLang)		
		480 × 480	600 × 600	720 × 720	1	8	2 × 8
LLaMA3-8B	WaferLLM	20320.6	25037.2	27686.5	13988.3	17361.6	13994.2
	T10	175.0	156.6	132.8			
	Ladder	61.8	42.3	31.3			
LLaMA2-13B	WaferLLM	13685.1	16854.2	17498.3	7805.1	12287.1	/ ³
	T10	121.3	100.6	81.3			
	Ladder	47.3	33.1	24.2			
CodeLLaMA-34B	WaferLLM	5471.4	7540.1	8526	5382.5	7155.5	6409.2
	T10	49.1	46.8	41.2			
	Ladder	30.1	23.1	17.7			
QWen2-72B	WaferLLM	2785.2	3775.5	4421.6	1677.3	3803.8	3750.5
	T10	24.9	23.5	21.5			
	Ladder	16.8	12.8	10.1			

表 3: 预填充阶段的每请求吞吐量 (TPR)

模型	方法	WSE-2 核心			A100 GPU 数 (SGLang)		
		420 × 420	540 × 540	660 × 660	1	8	2 × 8
LLaMA3-8B	WaferLLM	2699.9	2501.5	2243.3	78.9	260.4	164.6
	T10	418.3	339.4	265.1			
	Ladder	14.6	13.1	11.4			
LLaMA2-13B	WaferLLM	2039.2	1899.4	1739.8	48.7	175.8	/ ³
	T10	341.8	270.8	233.7			
	Ladder	11.0	9.9	9.0			
CodeLLaMA-34B	WaferLLM	1450.8	1407.7	1359.2	26.1	100.4	84.5
	T10	278.2	222.4	193.1			
	Ladder	6.1	6.2	5.8			
QWen2-72B	WaferLLM	839.7	824.3	787.1	10.6	51.2	48.7
	T10	168.5	133.0	114.6			
	Ladder	3.2	3.3	3.4			

表 4: 解码阶段的每请求吞吐量 (TPR)

列生成任务, WaferLLM 平均加速 625×, 最高 677×; 对于上述较长序列生成任务, 平均加速 312×, 最高 342×。原因在于 Ladder 面向共享内存架构设计, 没有考虑 PLMR 设备特性, 因而无法把 LLM 划分到数百万核心 (P), 会产生代价高昂的长距离 NoC 通信 (L), 也无法处理本地内存约束 (M) 和有限路由资源 (R)。

预填充吞吐量. 表 3 给出了输入序列长度为 4096、核心配置从 480×480 到 720×720 时的预填充性能。CodeLLaMA-34B 与 QWen2-72B 超出 WSE-2 和单块 A100 的内存容量, 因此我们评测其中若干层, 并依据各层结构均匀这一特点按比例缩放结果。

WaferLLM 通过有效处理全部 PLMR 属性, 获得了显著加速: 相比 T10 平均约 160×、最高 178×; 相比 Ladder 约 270–450×。如第 2 节所述, GEMM 是主要瓶颈, 而 MeshGEMM 显著提升了 WaferLLM 的预填充性能; 第 7.2 节将对此作详细分析。

此外, WaferLLM 在所有模型上都能随核心数增加而提升吞吐量。例如, 当核心规模从 480 × 480 扩大到 720 × 720 时, QWen2-72B 提升 1.6×, LLaMA3-8B 提升 1.4×。相反, T10 和 Ladder 无法有效扩展, 增加核心后吞吐量甚至会下降。主要原因是 T10 与 Ladder 没有考虑高度非均匀分布式内存的空间局部性 (L), 因而可能频繁跨核心远距离读取数据, 增加通信开销。

解码吞吐量. 表 4 给出了核心配置从 420×420 到 660×660 时的解码吞吐量。对于 CodeLLaMA-34B 和 QWen2-72B

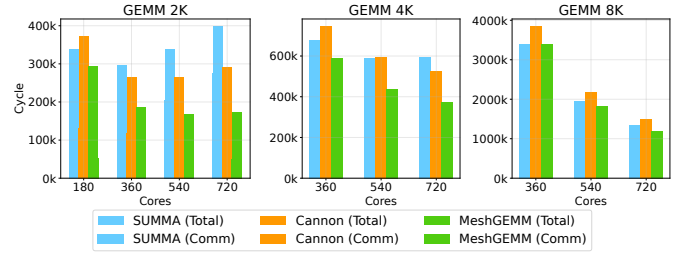


图 9: MeshGEMM 与 SUMMA、Cannon 的比较

在 WSE-2 与单块 A100 上的解码基准, 我们同样评测若干层并按比例缩放结果。

通过处理全部 PLMR 属性, WaferLLM 相比 T10 平均加速 5.7×、最高 6.5×; 相比 Ladder 平均加速 217×、最高 260×。

WaferLLM 在预填充阶段相对 T10 加速 160×, 但在解码阶段只加速约 6.5×; 相比之下, 它在预填充和解码阶段都能稳定保持相对 Ladder 的 200–500× 优势。主要原因是, 预填充涉及 dist-GEMM 计算, 要求每个数据元素按照固定顺序穿过同一行或同一列上的核心; 解码中的 dist-GEMV allreduce 则只要求遍历同一行或同一列的核心, 并不强制严格访问顺序。T10 的 compute-shift 范式虽然没有考虑网格 NoC 上核心的空间局部性, 但仍考虑了分布式内存, 所以当内存 (通信) 访问与顺序无关时仍可获得性能收益。Ladder 则完全针对共享内存架构设计, 无论是否要求内存访问顺序, 都会产生严重通信开销。

与 SGLang 比较. 如表 2、3 和 4 所示, 从 8B 到 72B 的多种模型上, WaferLLM 在预填充、解码和端到端性能方面都明显优于 SGLang 与 A100 GPU 集群。第 7.5 节将进一步讨论与 GPU 的比较。

值得注意的是, 与 SGLang 相比, T10 和 Ladder 未能利用 WSE-2 强大的硬件能力。例如, 在预填充任务以及 Ladder 的解码任务中, 它们的性能甚至低于单块 A100 上的 SGLang。这表明, 在晶圆级芯片上设计高性能系统与算法必须仔细考虑 PLMR 模型; 否则, 芯片的海量计算潜力不仅无法得到利用, 大规模网格 NoC 造成的高度非均匀分布式内存还可能使性能下降。

7.2 MeshGEMM

我们在 WSE-2 上, 针对不同核心规模与矩阵大小, 将 MeshGEMM 与 Cannon [6]、SUMMA [42] 进行比较。

扩展核心数. 图 9 表明, 对于不同大小的矩阵, MeshGEMM 在增加核心数时都能取得低于 SUMMA 和 Cannon 的延迟。相比 SUMMA 的流水线广播与 Cannon 的头到尾传输, MeshGEMM 的交错传输可以尽量降低每一步的通信开销, 并在大规模细粒度并行执行时尽可能重叠通信与计算。它展现出更强的可扩展性, 即使接近硬件

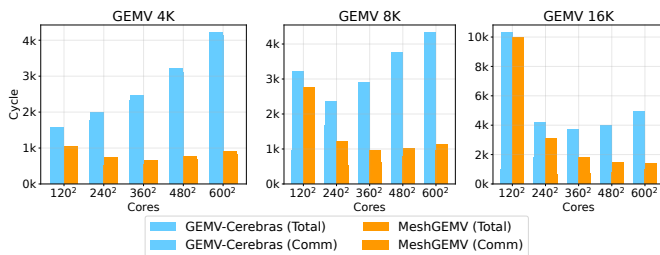


图 10: MeshGEMV 与 Cerebras GEMV 的比较

极限也能保持 70% 以上的计算效率。SUMMA 与 Cannon 的扩展性则较差：使用 720×720 个核心时，计算效率降至 50% 以下，主要原因是其通信开销随并行规模增长。

此外，增加核心数并不总能提高性能，小矩阵 GEMM 尤其如此。例如，对于 GEMM 2K，当核心规模从 360×360 扩大到 720×720 时，SUMMA 和 Cannon 的端到端延迟不降反升。原因是，核心数超过某个阈值后，单核计算成本只会次线性下降（受函数调用、逻辑检查等固定开销影响），同时通信开销却线性增长，如图 9 中阴影区域所示，最终使总体性能恶化。相比之下，MeshGEMM 在 720×720 个核心上仅略微增加通信开销，并保持稳定的端到端延迟。这是因为交错通信把每一步的通信成本限制为与核心数无关的常数（所有 dist-GEMM 算法的总通信开销仍会因步数增加而增长）。因此，即使在大规模细粒度并行下——每个核心只负责少量数据——MeshGEMM 也能更好地重叠通信与计算。

扩展矩阵大小。 我们还使用更大的矩阵评测 MeshGEMM，使 GEMM 成为计算强度更高的运算。在大规模下，通信成本的重要性虽然降低，MeshGEMM 仍保持可扩展性，并以较大优势优于 SUMMA 和 Cannon，把总周期减少约 17%。

图 9 中一个有趣现象是：对于 GEMM 8K，通信周期会随核心数增加而减少。这是因为处理大量数据时，通信受带宽而非延迟限制。增加核心数不仅提高聚合算力、降低计算开销，还增加聚合带宽，从而降低总体通信成本。

7.3 MeshGEMV

我们在不同核心规模和矩阵形状下，将 MeshGEMV 与 Cerebras 的默认 GEMV 实现（流水线 allreduce）进行比较。

通信延迟瓶颈。 图 10 表明，通信已成为 WSE-2 上 dist-GEMV 的主要瓶颈；当并行规模相对单核计算量很大时，通信开销甚至可占总开销的 90% 以上。不过，与 Cerebras 基线 GEMV 相比，MeshGEMV 显著降低了通信开销，端到端性能提高约 4.6 $\times$ 。原因是大规模 dist-GEMV 需要长距离 allreduce 通信，而 MeshGEMV 使用 K 树 allreduce，

模型	LLaMA3-8B	LLaMA2-13B
基于拼接 (PagedAttention)	382	16
基于移位 (WaferLLM)	137548	6168

表 5: 最大解码输出长度

GEMV	[1,16K] $\times$ [16K,16K]			[1,32K] $\times$ [32K,32K]		
SGLang (A100) 的 TP	1 GPU	8 GPU	2 $\times$ 8 GPU	1 GPU	8 GPU	2 $\times$ 8 GPU
时间 (ms)	0.336	0.253	0.340	1.231	0.341	0.339
MeshGEMV (WSE-2)	0.0012			0.00203		
A100/WSE-2 能耗比	7.47	44.97	120.88	16.17	35.83	71.25

表 6: 比较 MeshGEMV (WSE-2) 与 SGLang (A100) 张量并行的 GEMV 延迟和能耗。

最大化 allreduce 过程的并行度，从而尽量降低关键路径上的通信与计算成本。

扩展核心数和矩阵大小。 我们还用更大的矩阵评测 MeshGEMV。与 2K 相比，基线在 GEMV 8K 和 16K 上的端到端开销呈现先降后升的趋势。最初的下降来自核心数增加：更高的聚合算力与带宽降低了计算和通信成本；但核心数继续增长后，allreduce 通信延迟最终超过计算与带宽，成为主要瓶颈。MeshGEMV 的拐点出现得更晚，这意味着对于相同数据规模，随核心数增长，其通信开销的增速远低于基线，表现出更好的可扩展性。这使片上映射策略更灵活，例如可以用更多核心存储额外 LLM 参数，以满足内存限制 (M)，同时不引入显著额外开销。

7.4 基于移位的 KV cache 管理

我们还将基于移位的 KV cache 管理与 PagedAttention 中基于拼接的方案进行比较。实验使用 LLaMA3-8B 与 LLaMA2-13B，并采用第 7.1 节端到端推理评测的相同设置来测量 KV cache 容量。表 5 表明，对于 LLaMA3-8B 和 LLaMA2-13B，WaferLLM 基于移位的 KV cache 管理分别比拼接方案多支持 $360\times$ 和 $385\times$ 的 token。该提升来自均衡的核心利用率，以及移位方案对数据倾斜问题的消除。

7.5 与 GPU 比较

我们使用 Cerebras WSE-2 和 NVIDIA A100，将 WaferLLM 与 GPU 上最先进的 LLM 推理系统进行比较。若要与 H100 公平比较，则需要使用同为 5nm 工艺的 WSE-3，但我们无法获得该设备。

GEMV。 我们将 MeshGEMV 与遵循 SGLang [53] 多 GPU 张量并行的 GEMV 并行策略进行比较；每块 GPU 上的计算使用 cuBLAS 加速。如表 6 所示，针对不同矩阵大小，MeshGEMV 比单块 A100 GPU 快 280–606 $\times$ ，展示了晶圆级设备提供巨大内存带宽的优势。其能效也高出 7.5–16 $\times$ ，体现了晶圆内连接（连接片上内存）相对 GPU

预填充 (4K 上下文)	LLaMA3-8B			LLaMA2-13B	
	1 GPU	8 GPU	2 × 8 GPU	1 GPU	8 GPU
SGLang (A100) TPR	13988	17361	13994	7805	12287
WaferLLM WSE-2 TPR	27686			17498	
A100/WSE-2 能耗比	0.05	0.34	0.84	0.06	0.30

表 7: 比较 WaferLLM (WSE-2) 与 SGLang (A100) 的预填充吞吐量 and 能耗。

解码 (4K 上下文)	LLaMA3-8B			LLaMA2-13B	
	1 GPU	8 GPU	2 × 8 GPU	1 GPU	8 GPU
SGLang (A100) TPR	78	260	164	48	175
WaferLLM WSE-2 TPR	2700			2039	
A100/WSE-2 能耗比	0.92	2.22	7.02	1.13	2.49

表 8: 比较 WaferLLM (WSE-2) 与 SGLang (A100) 的解码吞吐量和能耗。

中基于 PCB 的连接 (连接片外 HBM) 的优势。

多 GPU 分布式 GEMV 的可扩展性有限: 单块 A100 的每 FLOP 能效最高; 从一块扩展到八块 GPU 时, 性能仅提高 1.32 $\times$, 扩展到十六块后反而下降。这种低效率源于内存密集性, 以及 NVLink 和 IB 上的通信开销。相比之下, MeshGEMV 利用 WSE-2 的低延迟 NoC, 在数十万个核心上扩展; 与 GPU 集群的最佳结果相比, 峰值性能高出 166–210 $\times$, 能效高出 45–70 $\times$ 。此外, 在两台机器共 16 块 GPU 上, 16K 与 32K GEMV 的性能相近, 主要是因为跨两节点分布式 GEMV 的通信开销主导了计算。

尽管有这些优势, MeshGEMV 仍未达到理论上的 7,000 $\times$ 提升。性能剖析发现有三个原因: (i) 仍处于第二代的 WSE-2 核心无法完全重叠内存访问与计算; (ii) 边缘核心利用不足; (iii) 尽管 MeshGEMV 已有效缓解, NoC 长距离通信开销依然存在。我们预计, 随着晶圆级加速器成熟, 这些差距会继续缩小。

LLM 推理. GPU 集群可达到的最大 TPR 显著低于 WSE-2。我们把 WaferLLM 与多 A100 GPU 集群上的 SGLang 进行比较后发现, 在多种输入/输出长度以及从 8B 到 72B 的模型规模上, WaferLLM 的 TPR 高出 6–20 $\times$; 输出越长、模型越大, 差距越明显 (表 2)。如表 7 和表 8 所示, SGLang 从 1 块扩展到 8 块 GPU 时, 预填充仅加速 1.2–1.6 $\times$, 解码仅加速 3.3–3.6 $\times$, 远低于理想的线性扩展。受节点间通信瓶颈影响, 扩展到 16 块 GPU 后性能进一步下降。因此, 稠密模型的峰值 TPR 通常出现在单个 8-GPU 节点内, 而 WaferLLM 相比这一配置仍最高快 20 $\times$ 。对于要求较高单请求吞吐量的场景, 晶圆级加速器的能力显著强于传统 xPU 系统。

能效方面, 虽然一颗 WSE-2 的面积约为 A100 的 47 $\times$, 功耗和成本约为 A100 的 37 $\times$, 但由于 GPU 的解码扩展不是线性的, WaferLLM 相比 SGLang 的最佳多

GPU 结果仍有 2–2.5 $\times$ 能效优势。对于测试时扩展等长输出场景, 这一优势尤其重要, 因为服务成本是关键因素。不过, 与单独的 GEMV 相比, WaferLLM 的解码能效优势有所缩小, 原因包括: (i) WSE-2 核心的本地 SRAM 只有 48KB, 妨碍高效张量并行, 并迫使系统采用流水线并行, 导致利用率最高降低 5 $\times$; (ii) 面向 GPU 优化的 LLaMA 模型具有较窄的层, 限制了 WSE-2 上的层放置并进一步降低利用率。

8 实现细节与未来方向

下面讨论 WaferLLM 和晶圆级加速器当前的局限, 并展望未来的解决方案。

硬件架构. 当前, WaferLLM 的性能受到流水线并行所引入执行气泡的限制。若把单核本地内存增加 5–6 $\times$, 就可以不再使用流水线并行, 转而像 vLLM 与 SGLang 一样完全采用张量并行。晶圆级芯片设计者已经沿这一方向推进。Cerebras WSE-3 保持相同的 NoC 配置, 但提升了单核效率和本地内存; Tesla Dojo 则为每个核心配备 1MB 内存。

内存/计算比. LLM 解码要求接近 1:1 的内存/计算比。然而, A100 等 GPU 的片上 SRAM 有限, 必须频繁访问片外内存, 因此 FP16 下的比例只有较差的 1:312。多 GPU 配置的大量 GPU 间通信会使这一问题进一步恶化。相比之下, Cerebras WSE-2 通过低延迟 NoC 把大部分模型权重映射到片上内存, 获得接近理想的局部性, 并趋近 1:1。要在基于网格的 NoC 上充分实现这种平衡, 必须遵守 PLMR 模型; 这使 WaferLLM 的 TPR 能够以数量级优势超过基于 GPU 的系统。

处理可靠性问题. 当前, Cerebras WSE-2/3 由硬件处理故障, 只向软件暴露按网格组织的健康核心, 因此软件层无需显式处理。制造时还会内置冗余核心与链路, 片上 SoC 在运行时动态重映射并绕过缺陷, 从而以较低冗余成本保证最小的性能影响。与此同时, 晶圆级芯片厂商近期报告其晶圆功能区域比例达到 93%, 高于商用 GPU 的 70–80% [8], 原因是每个核心的面积更小。我们在两年 WSE-2 部署期间观察到较高可靠性, 证实这些容错机制在真实使用中的有效性。

多种模型架构. WaferLLM 对 MoE 同样有益, 因为 MoE 与稠密 LLM 共享 MeshGEMM、MeshGEMV 和基于移位的 KV cache 管理等关键算子。主要差别是注意力层与专家层之间的 all-to-all 通信, 我们使用 WSE-2 的 NoC 多播操作实现该通信。卸载和稀疏注意力等针对稀疏模型的进一步优化, 是未来的研究方向。

超越 Cerebras WSE. 尽管评测在 Cerebras WSE 上完成, PLMR 模型仍可推广到 Tesla Dojo 等新兴类网格架构; 这些架构同样包含数十万个带本地内存的核心, 并受到 NoC 路由约束。2D 环面或网格-交换机混合拓扑等变体也符合 PLMR。我们的 MeshGEMM 与 MeshGEMV 面向最坏情况下的 2D 网格设计, 在这些平台上仍具竞争力。除片上网格外, Tenstorrent 核心级与板卡级网格中的芯片间网格互连也非常符合 PLMR。展望未来, 台积电预计到 2027 年晶圆级集成密度将提高 40 $\times$; 这类进展进一步强化了我们方法的长期适用性。

9 相关工作

深度学习框架与编译器. PyTorch、TensorFlow 和 XLA [1, 10, 28, 34, 36, 37, 45, 51, 54, 57] 等现有深度学习框架与编译器面向共享内存架构设计, 并采用基于 tile 的“加载-计算-存储”计算模型。该模型在共享内存上有效, 却忽略了 PLMR 设备的独特特性, 因此在晶圆级 AI 芯片上效率较低。vLLM 和 TensorRT-LLM [20, 56] 等框架已经出现以支持现代 LLM, 但它们仍依赖为共享内存架构设计的框架和编译器 (例如 PyTorch [34]), 因此在晶圆级芯片上继承了相似局限。

分布式 GPU 与 TPU 系统. 理论上, 可以把片上分布式内存架构视作一个分布式 LLM 系统, 已有工作对此进行了研究 [20, 23, 35, 49, 52, 56]。然而, 这些系统面向 GPU 或 TPU pod (最多数千个节点) 设计, 依赖能力更强的路由器, 并且没有本地内存约束, 因此与 PLMR 模型不匹配。这些方法与本文关注的片上扩展互为补充。

脉动阵列. Amazon Trainium 和 Google TPU 等 AI 加速器采用的脉动阵列架构 [22], 主要关注小型核心设计, 而不是更大的晶圆级加速器。单个核心中的处理单元数量有限 (通常最多数百个), 因此它们不是 PLMR 设备, 但可以与 WaferLLM 互补。例如, Cerebras WSE 核心可以使用脉动阵列加速本地 GEMM。

数据流架构. 以往研究探索了在考虑核心间连接的数据流架构上执行计算。TENET [26] 以数据流模式把计算映射到互连核心的空间与时间维度; DISTAL [50] 支持用数据流方法调度分布式集群; SambaNova [14] 结合模型并行与流水线并行来执行 DNN。然而, 这些工作都未把计算扩展到晶圆级芯片。

晶圆级 allreduce. 近期研究 [27] 已探索晶圆级 allreduce, 但单次 allreduce 无法像 WaferLLM 一样完整并行 GEMV, 也不能支持完整 LLM 推理。此外, 该工作是 WaferLLM 所提出 K 树 allreduce 的一个特例。

10 结论

我们把本文视为探索晶圆级计算在 LLM 中潜力的一项基础工作。简洁而有效的 PLMR 模型揭示了重要机会, 并指导我们开发出首个晶圆级 LLM 并行方案, 以及可在晶圆级加速器上扩展的 GEMM、GEMV 算法。尽管当前晶圆级设备的软件栈仍有限制, 本文方法在性能与能效上都取得了数量级提升。我们希望这项工作能促使研究者更加关注晶圆级计算, 并推动 AI 走向更可持续的未来。

11 致谢

我们衷心感谢 shepherd Marco Canini 和 OSDI 审稿人富有洞见的反馈, 这些意见显著改进了本文。我们也感谢 Yuqing Xia (微软) 在本工作开发期间提供的宝贵意见。

我们感谢 Edinburgh International Data Facility (EIDF) 和 Edinburgh Parallel Computing Centre (EPCC) 提供硬件资源, 尤其感谢它们对 Cerebras WSE 系统访问的支持。感谢 Nick Johnson 协调访问, 并感谢 Mark Parsons 支持硬件采购和本项目。

最后, 我们感谢 Cerebras 的 Leighton Wilson 与 Mathias Jacquelin 慷慨相助; 他们耐心而详尽地回答了我们关于 CSL 与 Cerebras WSE 的许多问题。

参考文献

- [1] M. Abadi, P. Barham, J. Chen, et al. TensorFlow: A system for large-scale machine learning. *OSDI 2016*, 2016.
- [2] Advanced Micro Devices. AMD optimizes EPYC memory with NUMA. White paper, Advanced Micro Devices, Inc., 2023.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023.
- [4] AMD. AMD XDNA adaptive architecture, 2023. Accessed: 2024-11-29.
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [6] Lynn Elliot Cannon. *A cellular computer to implement the kalman filter algorithm*. PhD thesis, Montana State University, 1969.
- [7] Cerebras Systems. GEMM with collective operations. Accessed: 2024-10-05.
- [8] Cerebras Systems. 100× defect tolerance: How cerebras solved the yield problem, 2022. Accessed: 2025-04-29.
- [9] Cerebras Systems. Benchmark GEMV collectives, 2023. Accessed: 2024-11-29.
- [10] T. Chen et al. TVM: An automated end-to-end optimization stack for deep learning. *SSP 2018*, 2018.
- [11] Jaeyoung Choi, Jack J. Dongarra, and David W. Walker. Parallel matrix transpose algorithms on distributed memory concurrent computers. *Parallel Computing*, 21(9):1387–1405, 1995.
- [12] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [14] Mark Harris. SambaNova’s new AI chip and the quest for efficiency, 2023. Accessed: 2024-11-29.
- [15] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Hanyu Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on GPUs. *arXiv preprint arXiv:2311.01282*, 2023.
- [16] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [17] Norman Jouppi, Cliff Young, et al. Tensor processing units for machine learning: An introduction. Technical report, Google Inc., 2017.
- [18] Patrick Kennedy. Tenstorrent Blackhole and Metalium for standalone AI processing, 2024. ServeTheHome, Hot Chips 2024 Coverage.
- [19] Jinwoo Kim, Venkata Chaitanya Krishna Chekuri, Nael Mizanur Rahman, Majid Ahadi Dolatsara, Hakki Mert Torun, Madhavan Swaminathan, Saibal Mukhopadhyay, and Sung Kyu Lim. Chiplet/interposer co-design for power delivery network optimization in heterogeneous 2.5-d ICs. *IEEE Transactions on Components, Packaging and Manufacturing Technology*, 11(12):2148–2157, 2021.
- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [21] Mark LaPedus. TSMC bets big on advanced packaging, 2023. Accessed: 2024-11-29.

- [22] Ching-Jui Lee and Tsung Tai Yeh. ReSA: Reconfigurable systolic array for multiple tiny DNN tensors. *ACM Transactions on Architecture and Code Optimization*, 21(3):43:1–43:24, 2024.
- [23] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, 2023.
- [24] Sean Lie. Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning. *IEEE Micro*, 43(3):18–30, 2023.
- [25] Yiqi Liu, Yuqi Xue, Yu Cheng, Lingxiao Ma, Ziming Miao, Jilong Xue, and Jian Huang. Scaling deep learning computation over the inter-core connected intelligence processor with T10. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 505–521, 2024.
- [26] Liqiang Lu, Naqing Guan, Yuyue Wang, Liancheng Jia, Zizhang Luo, Jieming Yin, Jason Cong, and Yun Liang. TENET: A framework for modeling tensor dataflow based on relation-centric notation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 720–733, 2021.
- [27] Piotr Luczynski, Lukas Gianinazzi, Patrick Iff, Leighton Wilson, Daniele De Sensi, and Torsten Hoefer. Near-optimal wafer-scale reduce. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing, HPDC '24*, page 334–347. ACM, June 2024.
- [28] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. Rammer: Enabling holistic deep learning compiler optimizations with rTasks. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 881–897, 2020.
- [29] Xiaoning Ma, Qinzhi Xu, Chenghan Wang, He Cao, Jianyun Liu, Daoqing Zhang, and Zhiqiang Li. An electrical-thermal co-simulation model of chiplet heterogeneous integration systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 32(10):1769–1781, 2024.
- [30] Meta AI. Introducing MTIA: Meta’s next-generation training and inference accelerator for AI, 2024. Accessed: 2024-12-10.
- [31] Microsoft Azure. Azure Maia: For the era of AI from silicon to software to systems, 2023. Accessed: 2024-11-29.
- [32] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.
- [33] OpenAI. Openai o3 and o4-mini system card. <https://openai.com/index/o3-o4-mini-system-card/>, 2025. Accessed: 2025-05-03.
- [34] A. Paszke, S. Gross, S. Chintala, et al. Automatic differentiation in PyTorch. *NIPS 2017*, 2017.
- [35] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.
- [36] J. Rock et al. XLA: Optimizing TensorFlow for high performance. *Google Research*, 2017.
- [37] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 701–718, 2023.
- [38] Shukri J. Souri, Kaustav Banerjee, Amit Mehrotra, and Krishna C. Saraswat. Multiple Si layer ICs: motivation, performance analysis, and design implications.

In *Proceedings of the 37th Annual Design Automation Conference*, pages 213–220, 2000.

Performance Computer Architecture (HPCA), pages 1198–1210, 2022.

- [39] Cerebras Systems. Cerebras and g42 break ground on condor galaxy 3, an 8 exaflops ai supercomputer. <https://cerebras.ai/press-release/cerebras-g42-announce-condor-galaxy-3>, 2024. Accessed: 2025-05-03.
- [40] Cerebras Systems. Cerebras powers perplexity sonar with industry’s fastest ai inference. <https://www.cerebras.ai/press-release/cerebras-powers-perplexity-sonar-with-industrys-fastest-ai-inference>, 2025. Accessed: 2025-05-03.
- [41] Emil Talpes, Douglas Williams, and Debjit Das Sarma. DOJO: The microarchitecture of Tesla’s exa-scale computer. In *2022 IEEE Hot Chips 34 Symposium (HCS)*, pages 1–28, 2022.
- [42] R. A. Van De Geijn and J. Watts. SUMMA: scalable universal matrix multiplication algorithm. *Concurrency: Practice and Experience*, 9(4):255–274, 1997.
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008. Curran Associates, Inc., 2017.
- [44] James Wang. Cerebras brings instant inference to mistral le chat. <https://cerebras.ai/blog/mistral-le-chat>, 2025. Accessed: 2025-05-03.
- [45] Lei Wang, Lingxiao Ma, Shijie Cao, Quanlu Zhang, Jilong Xue, Yining Shi, Ningxin Zheng, Ziming Miao, Fan Yang, Ting Cao, et al. Ladder: Enabling efficient low-precision deep learning computing through hardware-aware tensor transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 307–323, 2024.
- [46] Tianqi Wang, Fan Feng, Shaolin Xiang, Qi Li, and Jing Xia. Application defined on-chip networks for heterogeneous chiplets: An implementation perspective. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1198–1210, 2022.
- [47] Wikipedia contributors. Static random-access memory, 2024. Accessed: 2024-12-10.
- [48] Wikipedia contributors. Wafer-scale integration, 2024. Accessed: 2024-12-10.
- [49] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654. ACM, 2024.
- [50] Rohan Yadav, Alex Aiken, and Fredrik Kjolstad. DISTAL: the distributed tensor algebra compiler. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 286–300, 2022.
- [51] Y. Zhao et al. Ansor: A compiler stack for auto-tuning tensor programs. *IEEE Transactions on Software Engineering*, 2020.
- [52] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuezhang Xu, Danyang Zhuo, Eric P Xing, et al. Alpa: Automating inter-and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.
- [53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37:62557–62583, 2024.
- [54] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. FlexTensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 859–873, 2020.

- [55] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024.
- [56] Yuxiao Zhou and Kecheng Yang. Exploring TensorRT to improve real-time inference for deep learning. In *2022 IEEE 24th International Conference on High Performance Computing & Communications*, pages 2011–2018. IEEE, 2022.
- [57] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Li-dong Zhou, Asaf Cidon, and Gennady Pekhimenko. ROLLER: Fast and efficient tensor compilation for deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 233–248, 2022.